REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

In this mega lab, we learn how to interface microprocessors with peripherals in the real world. This expands on our prior labs which were internal math and logic, to be able to take an input, store data, and do something functional and adaptable. This lab also introduces us to timers and their components (ticks, prescalers, effects of changing SCF), etc. In the future, we can and will use this to expand our functionality; likely with different, more complex peripherals, interrupts, to create literally ANYTHING our heart desires.

Crazy cool lab even if it was a bit painful! 😊

PRE-LAB EXERCISES

i. Which configuration register allows the utilization of a subset of I/O port pins to be configured as inputs (without affecting other pins)? Which configuration register allowed the utilization of a subset of I/O port pins to be configured as an output (without affecting other pins)?

The DIR register determines the direction of a GPIO port's pins. A port consists of 8 pins denoted pins 0 through 7. DIR is a binary number where each value represents the state of a pin, where a 1 signifies an output and an 0 signifies an input (which we mentioned in lecture was the OPPOSITE of what you would expect, 0 != **o**ut, 1 != **i**n.)

ii. What is the purpose of the SET/CLR/TGL variants of the DIR and OUT registers

The purpose is to have a way to control the data within a DIR/OUT register in a single clock cycle. These variants are the same width as the main register, and each bit corresponds to its primary register counterpart. A 1 in place of a bit in the SET register causes any of the corresponding bits to enter a high state, or output state; A 1 in place of a bit in the CLR register causes any of the corresponding bits to enter a low state, or input state; A 1 in place of a bit in the TGL register causes any of the corresponding bits to flip.

iii. Which I/O ports are utilized for the DIP switches and LEDs on the OOTB Switch & LED Backpack?

On the OOTB Switch & LED Backpack, the DIP switches are connected to Port A and LED's are connected to Port C. This is shown in **APPENDIX Figure 1**.

iv. Are the LEDs on the OOTB Switch & LED Backpack active-high, or active-low? Draw a schematic diagram for a single LED circuit with the same activation level used on the backpack, as well as one with the opposite activation level. Also, draw a schematic diagram for a single-pole, single-throw (SPST) switch circuit, using the same pull-up or pull-down resistor condition utilized on the backpack, as well as another switch circuit using the opposite configuration.

The LED's on the Switch & LED Backpack are **Active-Low**, seen in **APPENDIX Figure 1**, the drawn schematics are in **APPENDIX Figure 2**.

v. Would it be possible to interface the OOTB µPAD with an external input device consisting of 22 inputs? If so, describe how many I/O ports would be necessary. If not, explain why.

We have plenty of IO ports, including ports A, C, E, F as open for I/O. For 22 inputs, we would need at least three ports (with each port containing 8 I/O pins.)

vi. Assuming a system clock frequency of 2 MHz, a prescaler value of 64, and a desired timer/clock period of 44 ms, calculate a theoretically-corresponding timer/counter period value two separate times: once using a form of dimensional analysis, providing explanation(s) when appropriate, and another time using the general formula provided within The Most Common Use Case for Timer/Counters

$$\frac{2000000\ cycles}{1\ second} * \frac{1\ tick}{64\ cycles} = 31250\ ticks/second$$

PER = 0.044s * 31250 ticks/second = 1,375 ticks

$$PER = D\left(\frac{SCF}{PRE}\right) = 0.044\left(\frac{2000000}{64}\right) = 1{,}375\ ticks$$ 😊.

vii. Assuming a system clock frequency of 2 MHz, is a period of two seconds achievable when using a 16-bit timer/counter prescaler value of one? If not, determine if there exists any prescaler value that allows for this period under the assumed circumstances, and if there does, list such a value.

$$PER = D(\frac{SCF}{PRE})$$

$$PER = 2\left(\frac{2000000}{1}\right) = 4000000\ ticks$$

Too large for 16-bit counter (65535 ticks max)

$$PRE \geq D\left(\frac{SCF}{PER}\right) = 2\left(\frac{2000000}{65536}\right) = 61.035$$

Prescaler must be >= 61.035

viii. What is the maximum time value (to the nearest millisecond) representable by a timer/counter, if the relevant system clock frequency is 2 MHz? What about for a system clock frequency of 37.444 kHz?

The timers are 16-bit, so they can represent 2^16 – 1 counter increments, or 65535. The maximum pre-scaler for the ATxmega128A1U is 1024, so our largest time counted is:

$$(65{,}535 * 1024)\ /\ 2{,}000{,}000 \cong 33.554\ s\ at\ 2MHz$$
$$(65{,}535 * 1024)\ /\ 37{,}444 \cong 1792.219\ s\ at\ 37.444kHz$$

or 29 minutes and 52.219 seconds

ix. Create an assembly program to perform the same procedure as in § 3.2 but utilize a prescaler value of 2. Perform everything else described in the section for this new context, i.e., experimentally determine which whole number digital period value provides a corresponding period with the least amount of error, provide an appropriate screenshot of the relevant waveform with the minimal amount of error, including its precise frequency, and provide within the caption of the relevant screenshot the whole-number value that resulted in a minimal amount of error. Finally, describe and explain why there may be any differences between the two contexts, i.e., between using a prescaler value of 64, the value in exercise vi, and a prescaler value of 2.

$$PER = D(SCF/PRE)$$

$$PER = 0.044 * \left(\frac{2M}{2}\right) = 44{,}000\ ticks$$

Same program as Lab2_3 except `TC_CLKSEL_DIV64_gc -> TC_CLKSEL_DIV2_GC` and 44,000 ticks
In experiment we find that 88ms is best achieved by exactly 45,300 ticks.

x. Create an assembly program to keep track of elapsing minutes with a timer/counter, i.e., design a "watch" that only has a "minute-hand". (Hint: Instead of attempting to configure the period of the timer/counter to directly correspond to sixty seconds, configure the period to correspond to one second, and then keep track of how many times this timer/counter overflows [or underflows, if you wish to configure the timer/counter to count down].)

Done below, see exercises

xi. xi. It is stated above that, in the relevant context, it should not be necessary to debounce (nor wait for the release of) either tactile switch S2 on the OOTB MB or S1 on the SLB. Why is this so?

Because these jumps go to different modes, and are only jumped to when safe, so jumping to PLAY 3 times in a row would not cause an error as it is safely done. They do not cause data to be written multiple times and will not affect the usability.

xii. Provide a scenario in which the above program would experience unintended behavior due to tactile switch bouncing.

Even though our program is debounced, if it was not, we could store any number of duplicates of the same frame with a single press as it would do the store function multiples times. This is a challenge because of our processors immense clock speed, which causes it to run through our loops in nano to micro seconds.

PSEUDOCODE/FLOWCHARTS

LAB2_1:

Include def file

.cseg

.org 0x100

Rjmp MAIN:

MAIN:

Load r16 with a bitmask 0xFF

Store r16 as a clear bitmask to PORTA direction, setting it as an input

Store r16 as a set bitmask to PORTC direction, setting it as an output

Load r16 with PORTA's inputs

Store r16 to PORTC's output (PORTA's inputs -> PORTC's outputs)

Rjmp MAIN

LAB2_2:

Include def file

.cseg

.org 0x100

Rjmp MAIN

.org 0x0100

MAIN:

Load end of SRAM low bit to r16

Store to SPL

Load end of SRAM high bit to r16

Store to SPH

Load a single bit 0x01 to r16

Store to PORTC direction bitmask (turn one LED to output)

LOOP:

Toggle PORTC on/off

Call subroutine DELAY_10MS

Rjmp LOOP

DELAY_X_10MS:

Push r21/r22/r23 (incase we expanded, do not edit original data in registers)

Call subroutine DELAY_10MS DELAY_X times.

Pop r21/r22/r23 (in reverse order, LIFO)

return

Lab2_3 / Exercise ix:

Assumed we wanted a more accurate counter using the built in timer, so we can use prescalers.

.include defs

.cseg

.org 0

rjmp MAIN

.org 0x0100

MAIN:

Store low ramend to SPL

Store high ramend to SPH

Store 0x01 to PORTC_DIRSET (flash one LED)

Store low bit of 1374 to TCC0_PER

Store high bit of 1374 to TCC0_PER + 1

Store prescaler of 64 to TCC0_CTRLA

CHECK_LOOP:

Load TCC0 interrupt flags

Skip next if OVFIF is set

rjmp CHECK_LOOP

Store 0x01 to PORTC output toggle (turn it on/off)

Clear OVFIF

rjmp CHECK_LOOP

Exercise x: ; PER = D(SCF/PRE) = 1(2000000/128) = 15625 (experimentally was 2000?)

```
.include defs

.cseg

.org 0

rjmp MAIN


.org 0x0100

MAIN:

Store low ramend to SPL

Store high ramend to SPH

Set PC0,1 to outputs

Set period to 15625

Set prescaler to 1024


WATCH_LOOP: ;using r20 as seconds, r21 as minutes

Load flags

Skip if bit 0 set

rjmp WATCH_LOOP

clear flag

inc r20

Toggle LED1

cpi r20, 60

brne WATCH_LOOP

clr r20
```

inc r21

Toggle LED2

rjmp WATCH_LOOP

Lab2_4

```
.include “ATxmega128A1Udef.inc”

.equ ANIMATION_START_ADDR = 0x2000
.equ ANIMATION_SIZE = 0x2000

.dseg
.org ANIMATION_START_ADDR
ANIMATION:
Reserve ANIMATION_SIZE bytes

.cseg
.org 0
rjmp MAIN

.org 0x0100
MAIN:

Store low RAMEND to SPL
Store high RAMEND to SPH

Call IO_INIT
Call TC_INIT

Initialize X pointer to ANIMATION_START_ADDR ; playback pointer
Initialize Y pointer to ANIMATION_START_ADDR ; storage pointer
```

EDIT:

Check PLAY switch (S1 on the SLB, PF2)

If PLAY pressed

rjmp PLAY

Read DIP switch values

Load DIP switch values on LEDs

Check STORE FRAME switch (S2 on SLB, PF3)

If STORE FRAME switch not pressed

rjmp EDIT

Start debounce timer

Wait until timer overflow flag set

Stop timer

Clear overflow flag

Read STORE FRAME switch again

If STORE FRAME switch not pressed

rjmp EDIT

Store current DIP switch value into animation table using Y

Increment Y pointer

```
STORE_FRAME_SWITCH_RELEASE_WAIT_LOOP:

Wait until STORE FRAME (S2 on SLB, PF3) switch released
rjmp EDIT

PLAY:

Reload X pointer to ANIMATION_START_ADDR

PLAY_LOOP:

Read EDIT switch (S2 on OOTB MB, E1)
If EDIT switch pressed
rjmp EDIT

Compare X pointer with Y pointer

If X == Y
rjmp PLAY ; restart animation

Load frame from animation table using X
Output frame to LEDs
Increment X pointer

Set playback timer period
Start playback timer
```

```
PLAY_DELAY:

Read EDIT switch (S2 on OOTB MB, E1)

If EDIT switch pressed
rjmp EDIT

Wait until timer overflow flag set
Stop timer
Clear overflow flag
rjmp PLAY_LOOP

DONE:
rjmp DONE

IO_INIT:

Configure LED port as output
Configure DIP switch port as input
Configure SLB switches as input
Configure MB switch as input

ret
TC_INIT:

Configure debounce timer period
Configure playback timer period
```

```
Disable timer clock

Clear pending overflow flags


ret
```

PROGRAM CODE

Lab2_1:

```
;
; lab2_1.asm
;
; Created: 5/28/2026 10:20:43 AM
; Author : Evan Baesler

; ABSTRACT: This program functions as an endless loop, outputting
;           the values of a our DIP switch circuits on the OOTB uPAD
;           to their respective LED's, if an input switch is closed
;           the LED illuminates, vice versa

; NOTES: This program is made to be used with the OOTB uPAD and the
;        Switch and LED backpack.

; OBJECTIVES:
; (1) Designate I/O, the uPAD utilizes PA for our DIP switches, and PC
;     for our LED circuits
; (2) For active-low LED's, we must turn the output to 0 when we want
;     illumination. Which given the constraints of the switch being
;     closed, means we must have the LED read a 0 when the switch
;     outputs a 1.

.include "ATxmega128A1Udef.inc"

.cseg
.org 0x100
rjmp MAIN

MAIN:

ldi r16, 0b11111111 ; Loading a mask to cover PA/PC as input/output
sts PORTA_DIRCLR, r16 ; Sets DIR for PORTA pins to 0b00000000, making
                      ; them all inputs

sts PORTC_DIRSET, r16 ; Sets DIR for PORTC pins to 0b11111111 making
                                    ; them all outputs

lds r16, PORTA_IN

sts PORTC_OUT, r16

rjmp MAIN
```

Lab2_2:

```
;
; lab2_2.asm
;
; Created: 5/28/2026 12:43 PM
; Author : Evan Baesler

; ABSTRACT: This program functions as a simple way to do a flashing LED
;           with a 10ms period for on/off, with a 50% duty cycle.

; NOTES: This program is made to be used with the OOTB uPAD and the
;        Switch and LED backpack. We are configuring the stack pointer
;        to the highest data memory address,

; OBJECTIVES:
; (1) Designate I/O, the uPAD utilizes PC for our LED circuits
; (2) For active-low LED's, we must turn the output to 0 when we want
;     illumination. Which given the constraints of the switch being
;     closed, means we must have the LED read a 0 when the switch
;     outputs a 1.

.EQU DELAY_X = 0x01

.include "ATxmega128A1Udef.inc"

.cseg
.org 0
rjmp MAIN

.org 0x0100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16 ; We initialize the stack pointer to the end of our ram

ldi r16, high(RAMEND)
sts CPU_SPH, r16

ldi r16, 0x01 ; Loading a mask to cover PC as input

sts PORTC_DIRSET, r16 ; Sets DIR for PORTC pins to 0b11111111 making
                                    ; them all outputs

LOOP_MAIN:

sts PORTC_OUTTGL, r16

rcall DELAY_X_10MS

rjmp LOOP_MAIN

; SUBROUTINES BELOW

DELAY_40MS:

; 10 ms at 2MHz:
; 1 clock = 5us -> 10^-3/(5*10^-6) ~= 20000 clocks

push r21
```

```
push r22 ; push prior data of r21/r22 to prevent corruption of data
push r23 ; added to loop 10ms delay 4 times

ldi r23, 4 ; outer count

OUTER_LOOP:

ldi r22, 68 ; middle count

MIDDLE_LOOP: ; changed to middle loop after adding 4x count for 40ms delay

ldi r21, 100 ; Inner count

INNER_LOOP:

dec r21
brne INNER_LOOP ; dec & brne take 3 clocks total when branching
                            ; otherwise they take 2 clocks, with a count of 100
                            ; the branch is 99 * 3 + 1 * 2 clocks, or 299 clocks
                            ; 20000 / 299 ~= 66-67

dec r22
brne MIDDLE_LOOP ; loops = ~67
; 299 * 67) / (20000) = 1.00165 = 0.165% error, less than 3% allocated.

dec r23
brne OUTER_LOOP

pop r23
pop r22
pop r21 ; pop in reverse order, LIFO

ret

DELAY_X_10MS:

push r24
ldi r24, DELAY_X

LOOP_DELAY:

rcall DELAY_40MS
dec r24

brne LOOP_DELAY

pop r24

ret
```

Lab2_3:

```
/*
 * Lab2_3.asm
 *
 *  Created: 5/31/2026 5:23:56 PM
 *   Author: Evan Baesler
 *

 ABSTRACT: This program functions as an endless loop, outputting
           the values of a our DIP switch circuits on the OOTB uPAD
           to their respective LED's, if an input switch is closed
           the LED illuminates, vice versa

 NOTES: This program is made to be used with the OOTB uPAD and the
        Switch and LED backpack.

 OBJECTIVES:
 (1) Designate I/O, the uPAD utilizes PC for our LED circuits
 (2) For active-low LED's, we must turn the output to 0 when we want
     illumination. Which given the constraints of the switch being
     closed, means we must have the LED read a 0 when the switch
     outputs a 1.
 (3) We want to have a precise 44ms delay utilizing timer counters
     such as TCO/1, so we need to find the amount of ticks needed for
        44ms:

              PER = 0.044 * (2M / 64) = 1375 ticks -> 1375 - 1 = 1374
  */

.include "ATxmega128A1Udef.inc"

.cseg
.org 0
rjmp MAIN

.org 0x0100
MAIN:

; Stack Pointer Initialization:
ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16 ; Stack Pointer to RAMEND

; Output Designation:
ldi r16, 0x01
sts PORTC_DIRSET, r16 ; Sets 1 bit of PortC as output

ldi r16, low(1374) ; In testing found to be 0x055e
sts TCC0_PER, r16
ldi r16, high(1374)
sts TCC0_PER+1, r16

ldi r16, TC_CLKSEL_DIV64_gc ; Per doc8045, sets prescaler of 64
sts TCC0_CTRLA, r16

CHECK_LOOP:

lds r16, TCC0_INTFLAGS ; load the interrupt flags, section 14.12.10
```

```
sbrs r16, 0 ; skips the next line if the LSB (OVFIF) is set
rjmp CHECK_LOOP

ldi r16, 0x01
sts PORTC_OUTTGL, r16

sts TCC0_INTFLAGS, r16 ; flags can be cleared by writing a one

rjmp CHECK_LOOP
```

Exercise ix:

```
/*
 * Lab2_3.asm
 *
 *  Created: 5/31/2026 5:23:56 PM
 *   Author: Evan Baesler
 *

 ABSTRACT: This program functions as an endless loop, outputting
           the values of a our DIP switch circuits on the OOTB uPAD
           to their respective LED's, if an input switch is closed
           the LED illuminates, vice versa

 NOTES: This program is made to be used with the OOTB uPAD and the
        Switch and LED backpack.

 OBJECTIVES:
 (1) Designate I/O, the uPAD utilizes PC for our LED circuits
 (2) For active-low LED's, we must turn the output to 0 when we want
     illumination. Which given the constraints of the switch being
     closed, means we must have the LED read a 0 when the switch
     outputs a 1.
 (3) We want to have a precise 44ms delay utilizing timer counters
     such as TCO/1, so we need to find the amount of ticks needed for
        44ms:

              PER = 0.044 * (2M / 64) = 1375 ticks -> 1375 - 1 = 1374
  */

.equ TICKS = 45300

.include "ATxmega128A1Udef.inc"

.cseg
.org 0
rjmp MAIN

.org 0x0100
MAIN:

; Stack Pointer Initialization:
ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16 ; Stack Pointer to RAMEND

; Output Designation:
ldi r16, 0x01
sts PORTC_DIRSET, r16 ; Sets 1 bit of PortC as output

ldi r16, low(TICKS) ; In testing found to be 0x055e
sts TCC0_PER, r16
ldi r16, high(TICKS)
sts TCC0_PER+1, r16

ldi r16, TC_CLKSEL_DIV2_gc ; Per doc8045, sets prescaler of 64
sts TCC0_CTRLA, r16

CHECK_LOOP:
```

```
lds r16, TCC0_INTFLAGS ; load the interrupt flags, section 14.12.10

sbrs r16, 0 ; skips the next line if the LSB (OVFIF) is set
rjmp CHECK_LOOP

ldi r16, 0x01
sts PORTC_OUTTGL, r16

sts TCC0_INTFLAGS, r16 ; flags can be cleared by writing a one

rjmp CHECK_LOOP
```

Exercise x:

```
/*
 * ExerciseX.asm
 *
 *  Created: 5/31/2026 10:17:39 PM
 *   Author: Evan Baesler
 *

 ABSTRACT: This program functions as an endless loop, outputting
           a digital clock in seconds, and minutes (minutes go to
                 infinity, seconds reset every 60 seconds)

 NOTES: This program is made to be used with the OOTB uPAD and the
        Switch and LED backpack.

 OBJECTIVES:
 (1) Designate I/O, the uPAD utilizes PC for our LED circuits
 (2) Toggle PORTC LED0 for seconds, PORTC LED1 for minutes
 (3) Store value of seconds in R20 and value of minutes in R21

 */

 .equ TICKS = 2000 - 1

.include "ATxmega128A1Udef.inc"
.cseg
.org 0
rjmp MAIN

.org 0x0100
MAIN:
ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16
ldi r16, 0b00000011
sts PORTC_DIR, r16

clr r20
clr r21

ldi r16, low(TICKS) ; In testing found to be 0x055e
sts TCC0_PER, r16
ldi r16, high(TICKS)
sts TCC0_PER+1, r16

ldi r16, TC_CLKSEL_DIV1024_gc ; Per doc8045, sets prescaler of 64
sts TCC0_CTRLA, r16

WATCH_LOOP: ;using r20 as seconds, r21 as minutes

lds r16, TCC0_INTFLAGS ; load the interrupt flags, section 14.12.10
sbrs r16, 0 ; skips the next line if the LSB (OVFIF) is set
rjmp WATCH_LOOP

ldi r16, 0x01
sts TCC0_INTFLAGS, r16 ; flags can be cleared by writing a one

inc r20
```

```
ldi r16, 0x01
sts PORTC_OUTTGL, r16

cpi r20, 60
brne WATCH_LOOP

clr r20
inc r21

ldi r16, 0b00000010
sts PORTC_OUTTGL, r16

rjmp WATCH_LOOP
```

Lab2_4:

```
/*
 * Lab2_4.asm
 *
 *  Created: 5/31/2026 10:24:41 PM
 *   Author: Evan Baesler
 */

 ;***************************************************************************
;  File name: lab2_u26_4_skeleton.asm
;  Author: Christopher Crary
;  Last Modified By: Dr. Schwartz
;  Last Modified On: 20 May 2026
;  Purpose: To allow LED animations to be created with the OOTB uPAD,
;                   OOTB SLB, and OOTB MB.
;
;  NOTE: The use of this file is NOT required! This file is just given
;        as an example for how to potentially write code more effectively.
;***************************************************************************

;*******INCLUDES*************************************

; The inclusion of the following file is REQUIRED for our course, since
; it is intended that you understand concepts regarding how to specify an
; "include file" to an assembler.
.include "ATxmega128a1udef.inc"
;*******END OF INCLUDES*****************************

;*******DEFINED SYMBOLS*****************************
.equ ANIMATION_START_ADDR   =      0x2000 ; Start Address specified
.equ ANIMATION_SIZE                =      0x2000 ; 0x2000 - 0x3FFF
;*******END OF DEFINED SYMBOLS**********************

;*******MEMORY CONSTANTS*****************************
; data memory allocation
.dseg

.org ANIMATION_START_ADDR
ANIMATION:
.byte ANIMATION_SIZE
;*******END OF MEMORY CONSTANTS**********************

;*******MAIN PROGRAM*********************************
.cseg
; upon system reset, jump to main program (instead of executing
; instructions meant for interrupt vectors)
.org 0
       rjmp MAIN

; place the main program somewhere after interrupt vectors (ignore for now)
.org 0x0100          ; >= 0xFD
MAIN:
; initialize the stack pointer
       ldi r16, low(RAMEND)
       sts CPU_SPL, r16
       ldi r16, high(RAMEND)
       sts CPU_SPH, r16
; initialize relevant I/O modules (switches and LEDs)
       rcall IO_INIT
```

```
; initialize (but do not start) the relevant timer/counter module(s)
       rcall TC_INIT

; Initialize the X and Y indices to point to the beginning of the
; animation table. (Although one pointer could be used to both
; store frames and playback the current animation, it is simpler
; to utilize a separate index for each of these operations.)
; Note: recognize that the animation table is in DATA memory

       ldi XL, low(ANIMATION_START_ADDR)
    ldi XH, high(ANIMATION_START_ADDR)

    ldi YL, low(ANIMATION_START_ADDR)
    ldi YH, high(ANIMATION_START_ADDR)

; begin main program loop

; "EDIT" mode
EDIT:

; Check if it is intended that "PLAY" mode be started, i.e.,
; determine if the relevant switch has been pressed.

lds r16, PORTF_IN ; Read the input from S1 on the SLB to go to PLAY (PF2)

; If it is determined that relevant switch was pressed,
; go to "PLAY" mode.
sbrs r16, 2 ; active low, if the bit is set it is not pressed
rjmp PLAY ; skip if not pressed

; Otherwise, if the "PLAY" mode switch was not pressed,
; update display LEDs with the voltage values from relevant DIP switches
; and check if it is intended that a frame be stored in the animation
; (determine if this relevant switch has been pressed).

lds r17, PORTA_IN ; take inputs, place in outputs
sts PORTC_OUT, r17

sbrc r16, 3 ; Skip loop to EDIt if S2 on SLB is pressed, which is PF3

; If the "STORE_FRAME" switch was not pressed,
; branch back to "EDIT".

rjmp EDIT

; Otherwise, if it was determined that relevant switch was pressed,
; perform debouncing process, e.g., start relevant timer/counter
; and wait for it to overflow. (Write to CTRLA and loop until
; the OVFIF flag within INTFLAGS is set.)

ldi r16, low(156)
sts TCC0_PER, r16
ldi r16, high(156)
sts TCC0_PER+1, r16
; 5ms at DIV64, longest recorded bounce was 1.5ms, 3.5ms buffer

ldi r16, TC_CLKSEL_DIV64_gc
sts TCC0_CTRLA, r16

DEBOUNCE_LOOP:
```

```
lds r16, TCC0_INTFLAGS
sbrs r16, 0
rjmp DEBOUNCE_LOOP

; After relevant timer/counter has overflowed (i.e., after
; the relevant debounce period), disable this timer/counter,
; clear the relevant timer/counter OVFIF flag,
; and then read switch value again to verify that it was
; actually pressed. If so, perform intended functionality, and
; otherwise, do not; however, in both cases, wait for switch to
; be released before jumping back to "EDIT".

ldi r16, TC_CLKSEL_OFF_gc ; turn off timer
sts TCC0_CTRLA, r16

; clear OVFIF
ldi r16, 1 ; setting bit 0 clears OVFIF for TCC0
sts TCC0_INTFLAGS, r16

; read S2 on SLB (PF3) again to ensure it is still pressed; i.e. not bounce
lds r16, PORTF_IN
sbrc r16, 3 ; skip if it is still pressed
rjmp EDIT ; if not still pressed, stay in EDIT

; Store DIP switches to memory
lds r17, PORTA_IN
st Y+, r17

; Wait for the "STORE FRAME" switch to be released
; before jumping to "EDIT".
STORE_FRAME_SWITCH_RELEASE_WAIT_LOOP: ; this is S2 on the SLB; PF3 again

lds r16, PORTF_IN
sbrs r16, 3 ; skip if not pressed, i.e. action is done
rjmp STORE_FRAME_SWITCH_RELEASE_WAIT_LOOP

rjmp EDIT

; "PLAY" mode
PLAY:

; Reload the relevant index to the first memory location
; within the animation table to play animation from first frame.
      ldi XL, low(ANIMATION_START_ADDR)
      ldi XH, high(ANIMATION_START_ADDR)

PLAY_LOOP:

; Check if it is intended that "EDIT" mode be started
; i.e., check if the relevant switch has been pressed.`

lds r16, PORTE_IN

; If it is determined that relevant switch was pressed,
; go to "EDIT" mode.

; S2 on OOTB MB (E1)
sbrs r16, 1 ; Check if unpressed
rjmp EDIT
```

```
; Otherwise, if the "EDIT" mode switch was not pressed,
; determine if index used to load frames has the same
; address as the index used to store frames, i.e., if the end
; of the animation has been reached during playback.
; (Placing this check here will allow animations of all sizes,
; including zero, to playback properly.)
; To efficiently determine if these index values are equal,
; a combination of the "CP" and "CPC" instructions is recommended.

cp XL, YL ; compares the input/output tables
cpc XH, YH

; If index values are equal, branch back to "PLAY" to
; restart the animation.

breq PLAY

; Otherwise, load animation frame from table,
; display this "frame" on the relevant LEDs,
; start relevant timer/counter,
; wait until this timer/counter overflows (to more or less
; achieve the "frame rate"), and then after the overflow,
; stop the timer/counter,
; clear the relevant OVFIF flag,
; and then jump back to "PLAY_LOOP".

ld r17, X+ ; load from our animation table, post-increment
sts PORTC_OUT, r17 ; load to LEDs

ldi r18, low(1561)
sts TCC0_PER, r18
ldi r18, high(1561)
sts TCC0_PER+1, r18

ldi r18, TC_CLKSEL_DIV256_gc ; 1562 ticks with a prescaler of 256
sts TCC0_CTRLA, r18          ; ~= to 5Hz or 200ms period

PLAY_DELAY:
lds r16, PORTE_IN
sbrs r16, 1 ; S2 on OOTB MB = PE1, if pressed, QUIT
rjmp QUIT

lds r18, TCC0_INTFLAGS
sbrs r18, 0
rjmp PLAY_DELAY ; delay to 5Hz update speed

ldi r18, TC_CLKSEL_OFF_gc ; turn off timer
sts TCC0_CTRLA, r18

ldi r18, 1 ; clear flag
sts TCC0_INTFLAGS, r18
rjmp PLAY_LOOP

QUIT:

ldi r18, TC_CLKSEL_OFF_gc ; turn off timer
sts TCC0_CTRLA, r18
ldi r18, 1 ; clear flag
sts TCC0_INTFLAGS, r18
rjmp EDIT ; go back to edit mode
```

```
; end of program (never reached)
DONE:
	rjmp DONE
;*******END OF MAIN PROGRAM ************************

;*******SUBROUTINES*********************************

;***************************************************
; Name: IO_INIT
; Purpose: To initialize the relevant input/output modules, as pertains to the
;		application.
; Input(s): N/A
; Output: N/A
;***************************************************
IO_INIT:
; protect relevant registers

push r16

; initialize the relevant I/O

ldi r16, 0xFF
sts PORTC_DIRSET, r16 ; set all bits, making PORTC LED's outputs
sts PORTA_DIRCLR, r16 ; clr all bits, making PORTC switches inputs

ldi r16, 0b00001100 ; set relevant PORTF tactile switches to inputs
sts PORTF_DIRCLR, r16

ldi r16, 0b00000010 ; set relevant PORTE tactile switches to inputs
sts PORTE_DIRCLR, r16

; recover relevant registers

pop r16

; return from subroutine
	ret
;***************************************************
; Name: TC_INIT
; Purpose: To initialize the relevant timer/counter modules, as pertains to
;		application.
; Input(s): N/A
; Output: N/A
;***************************************************
TC_INIT:
; protect relevant registers

push r16

; initialize the relevant TC modules

ldi r16, low(156)
sts TCC0_PER, r16
ldi r16, high(156)
sts TCC0_PER+1, r16

; recover relevant registers

pop r16

; return from subroutine
```

```
       ret

;*******END OF SUBROUTINES*************************
```

APPENDIX

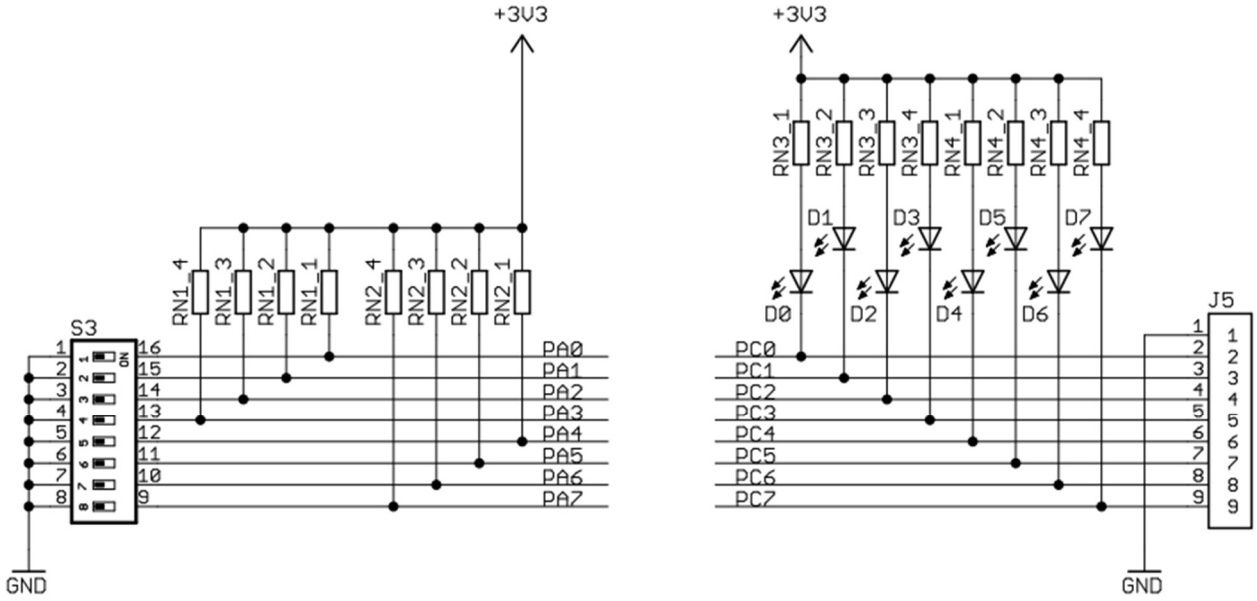

Figure 1: OOTB Switch & LED Backpack Schematic, active-low LED's.

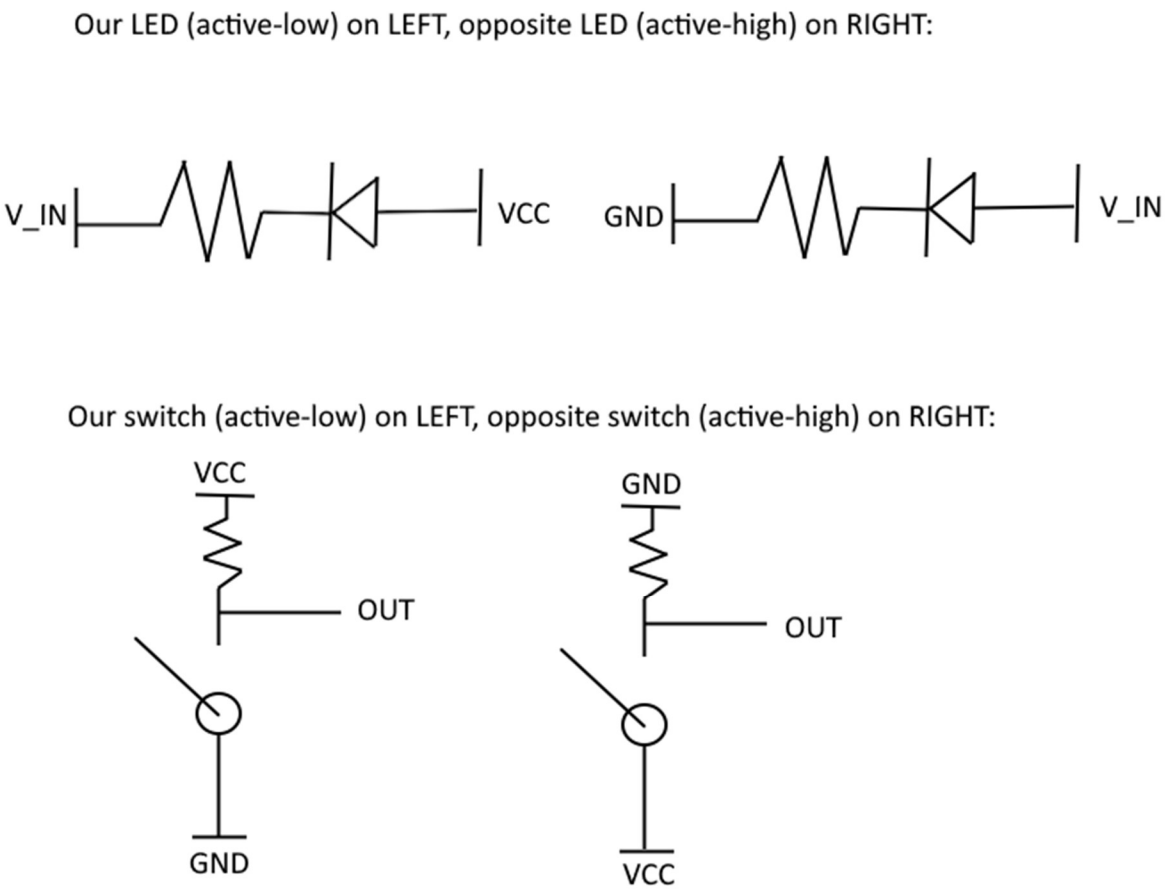

Figure 2: Active-low and active-high LED & switch circuits,

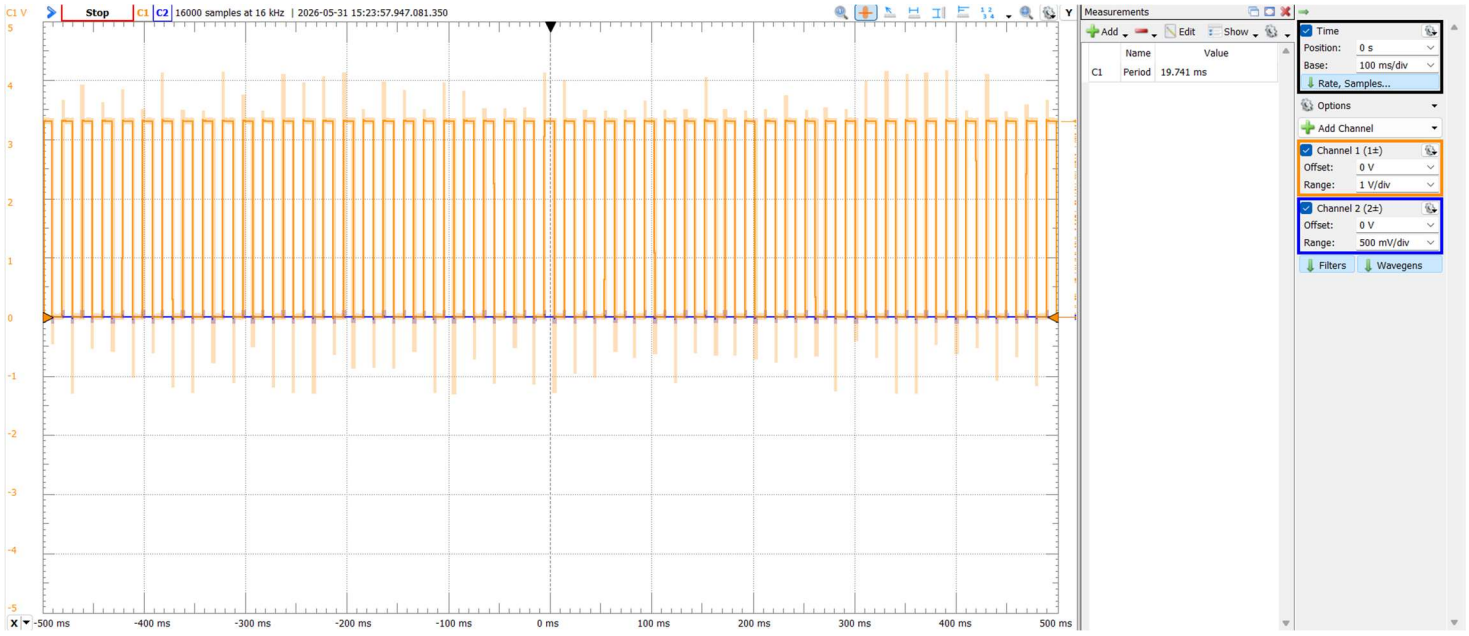

Figure 3: Lab2_2 period original period measurement through Waveforms.

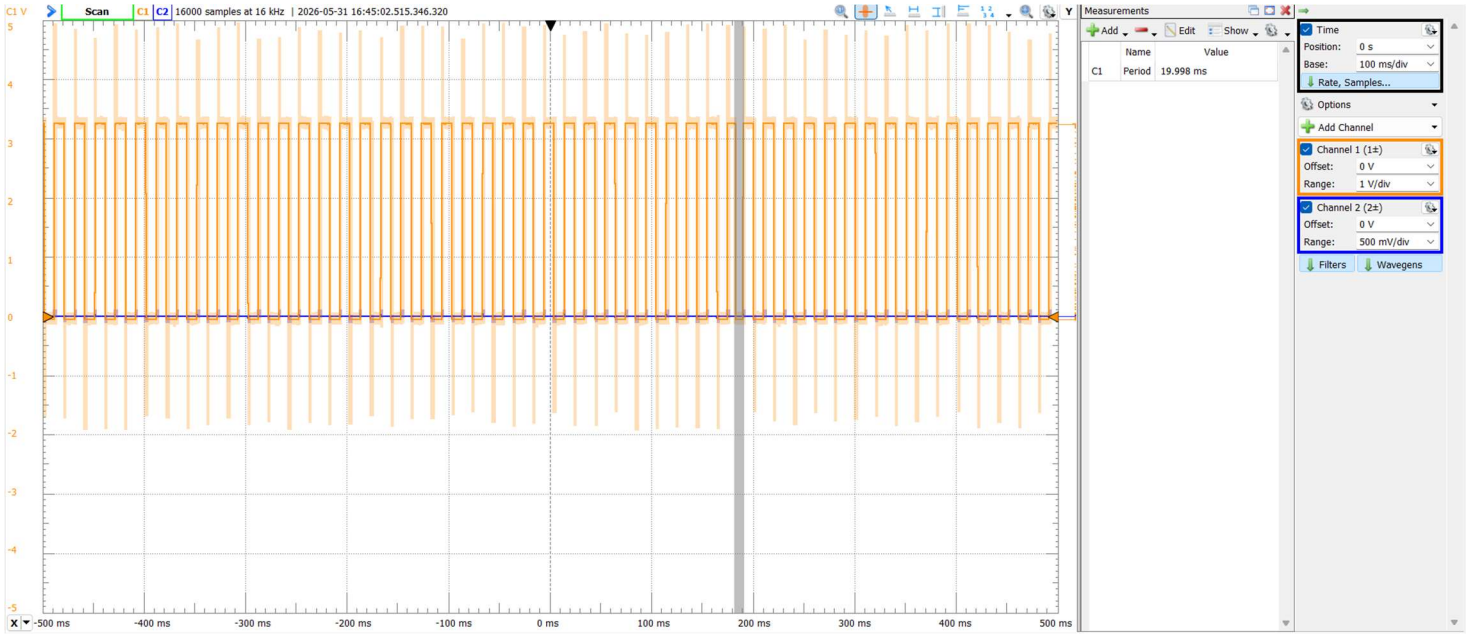

Figure 4: Lab2_2 corrected period measurement through Waveforms.

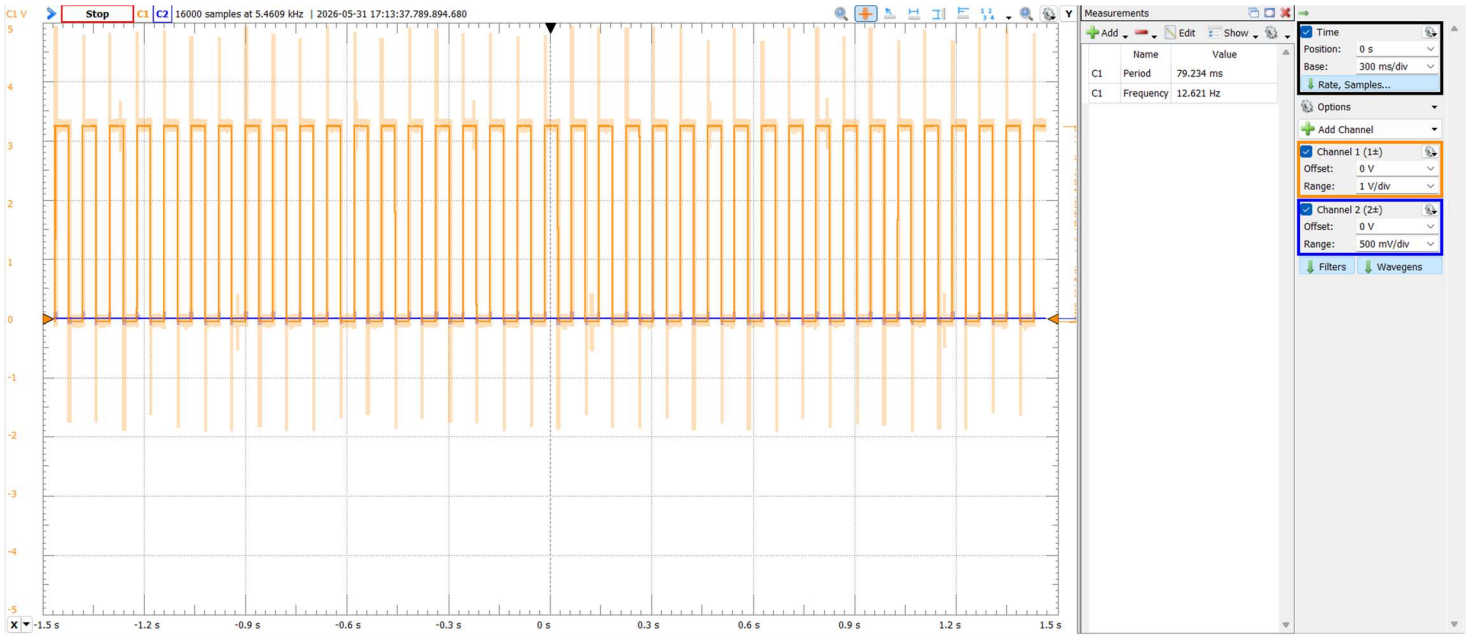

Figure 5: Lab2_2's ~40ms / ~12.5Hz delay in Waveforms.

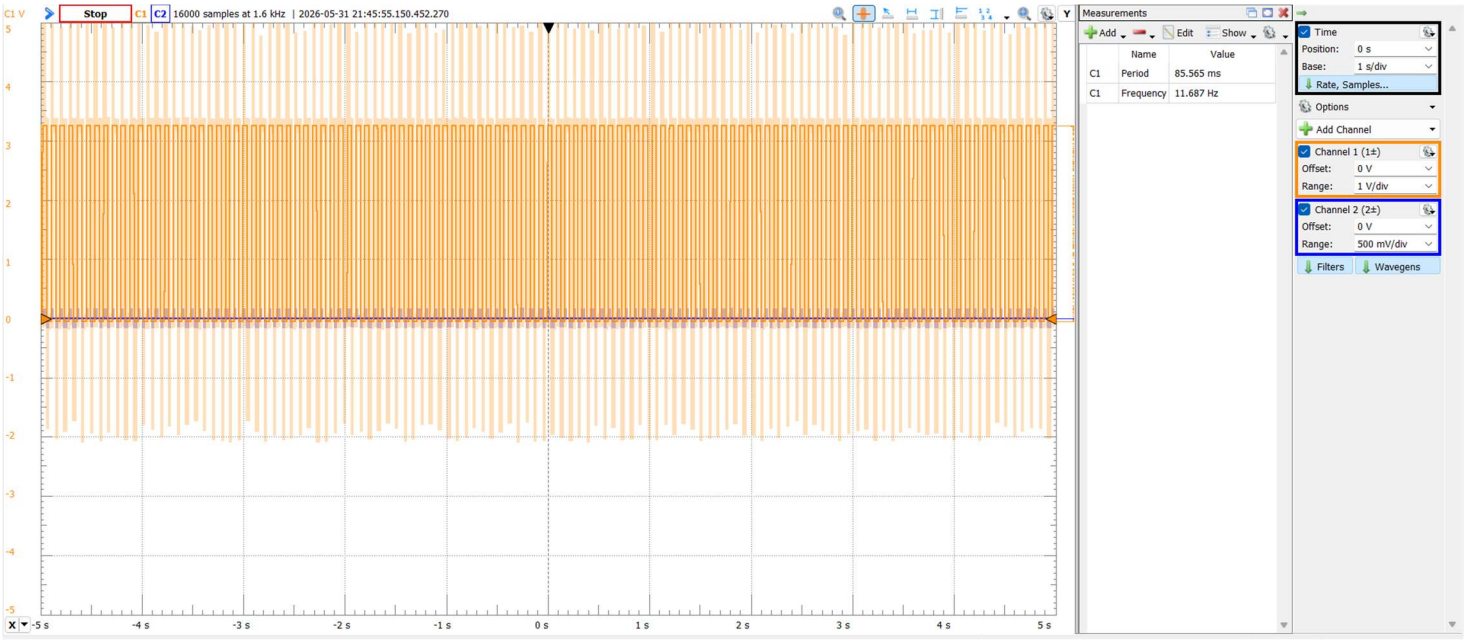

Figure 6: Calculated 88ms period square wave from TCC0 in Waveforms (1374 ticks).

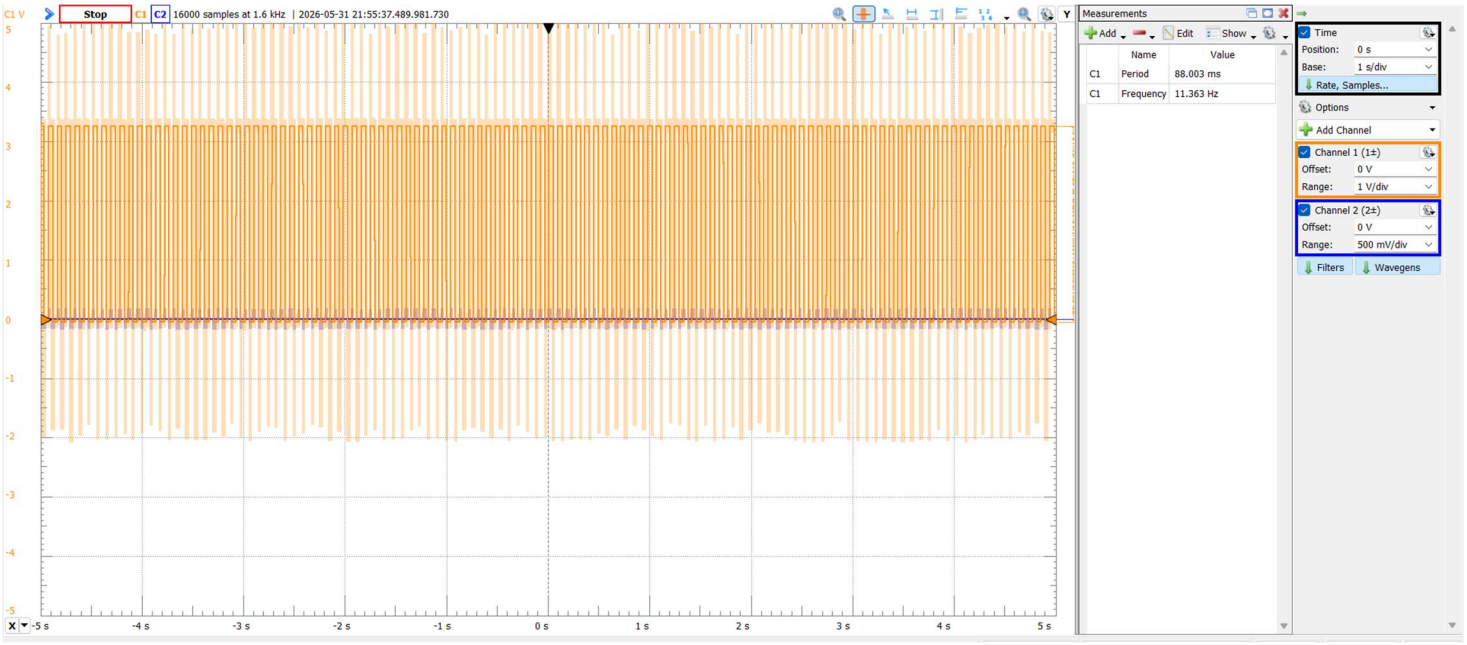

Figure 7: Experimental 88ms period square wave from TCC0 in Waveforms (1412 ticks).

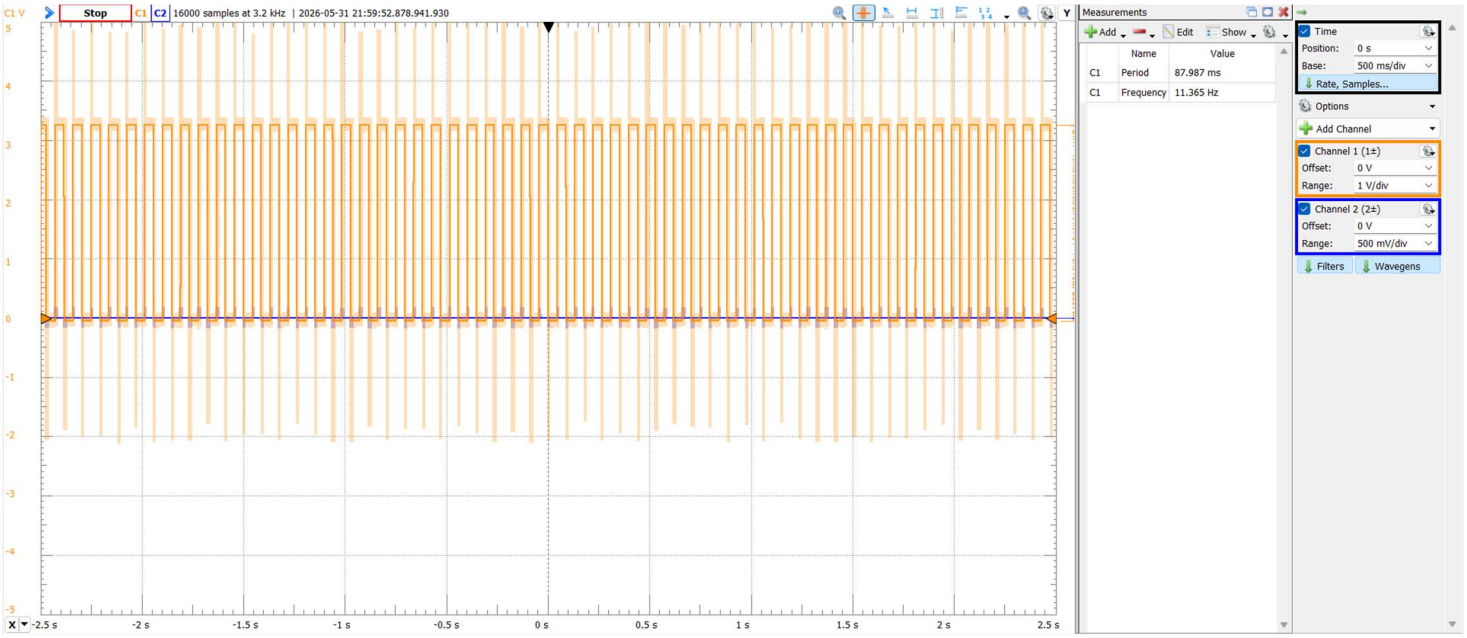

Figure 8: 88ms period square wave from TCC0 in Waveforms (DIV2, 45300 ticks).

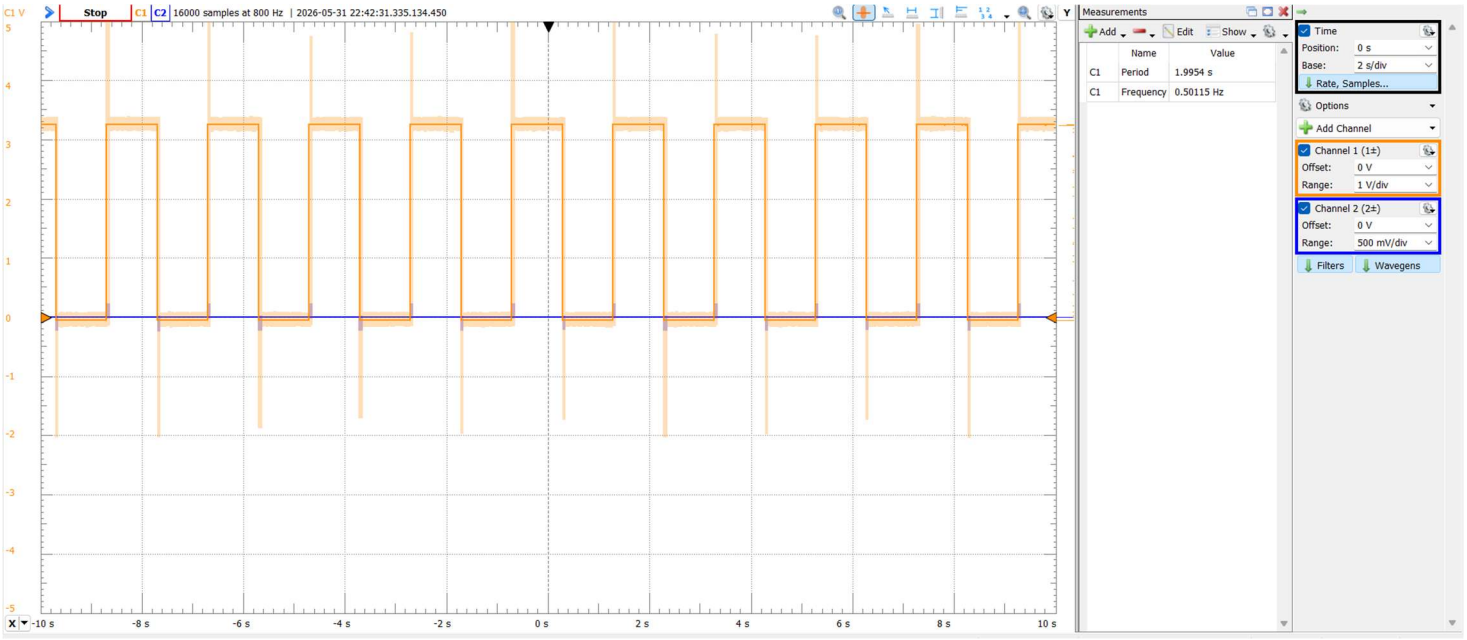

Figure 9: Exercise x clock exercise, seconds counter in Waveforms.0

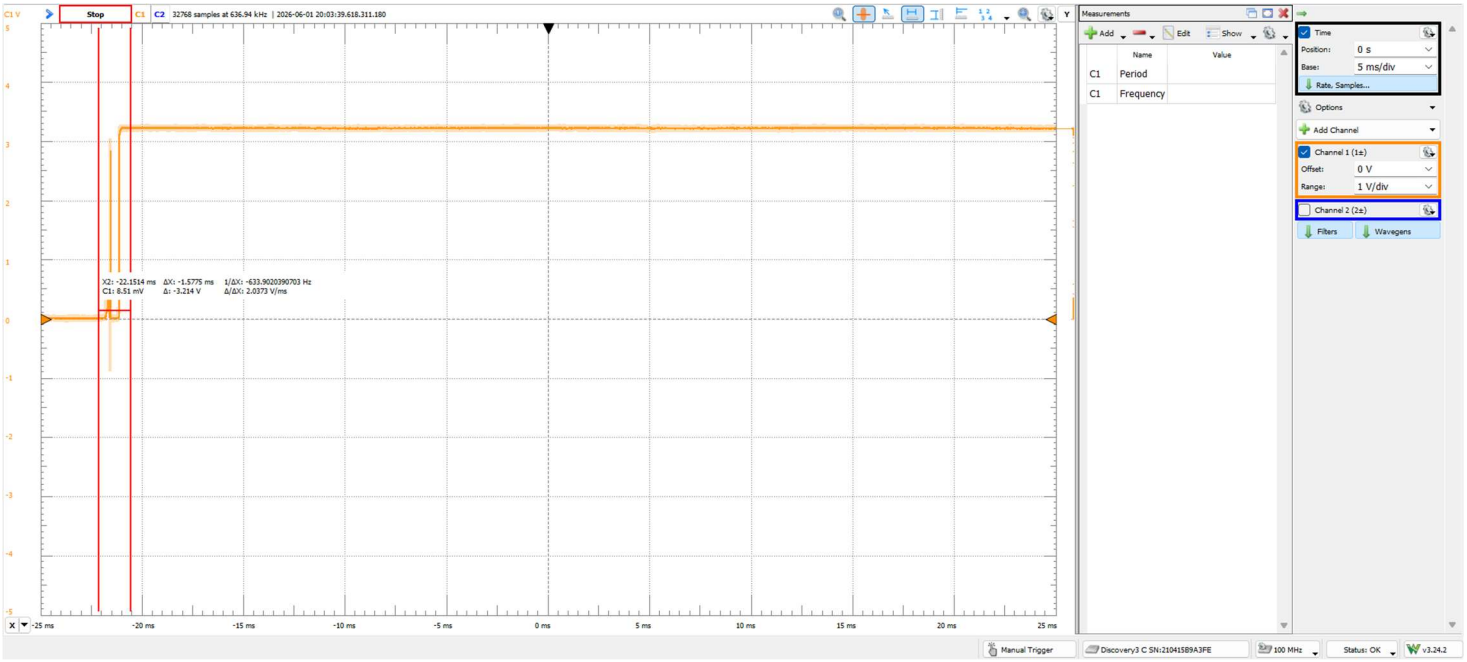

Figure 10: 1.5ms bounce, extremely hard to get, the debounce internal to the OOTB SLB is VERY good, I used a 2.5ms debounce incase.

Word Address			
ATxmega128A1U		**ATxmega64A1U**	
0		0	Application Section (bytes) (128K/64K)
			...
EFFF	/	77FF	
F000	/	7800	Application Table Section (bytes) (8K/4K)
FFFF	/	7FFF	
10000	/	8000	Boot Section (bytes) (8K/4K)
10FFF	/	87FF	

Figure 11: Program Memory Addressing

Byte Address	ATxmega64A1U
0	I/O Registers (4K)
FFF	
1000	EEPROM (2K)
17FF	
	RESERVED
2000	Internal SRAM (4K)
2FFF	
4000	External Memory (0 to 16MB)
FFFFFF	

Byte Address	ATxmega128A1U
0	I/O Registers (4KB)
FFF	
1000	EEPROM (2K)
17FF	
	RESERVED
2000	Internal SRAM (8K)
3FFF	
3000	External Memory (0 to 16MB)
FFFFFF	

Figure 12: Data Memory Addressing